

WEALTHORA

Personal Expense Tracker

Take Control of Every Taka.

FINAL ACADEMIC PROJECT REPORT

Course: CSE 215 - Object-Oriented Programming

Section: 11

Faculty: SAM3

Demonstration date: August 25, 2026

Registered topic: Personal Expenses Tracker

Technology: Java 25, Java Swing, Apache Ant, CSV persistence

Project team

Team member	Student ID
Shibli Rahman Moon	2534187012
Md. Nafij Jaman Rabbi	2513403642
Md. Monimul Haque	1821781042

Prepared for academic defense and submission review

Repository baseline: feature/shared-online-core @ d987a4a997fb

Document prepared: 2026-08-09

1. Problem statement, objectives and features

Personal finance records are often divided across notebooks, spreadsheets, wallet statements and memory. This makes it difficult to answer simple questions: how much money is available, where it was spent, whether a budget is at risk and which commitments are approaching. Wealthora addresses this problem as a local Java Swing Personal Expense Tracker that keeps day-to-day finance work understandable, testable and usable without a permanent internet connection.

Objectives

- Model income and expenses through production-used abstraction, inheritance, polymorphism, encapsulation and interfaces.
- Provide a coherent Swing workflow for accounts, transactions, transfers, categories, budgets, recurring entries, planning and reports.
- Protect local access through hashed credentials, verified email ownership, protected offline recovery, authorization and an exclusive data lock.
- Keep every authenticated user's finance records in a separate workspace and preserve them across normal restarts.
- Support safe import/export, backup/restore, presentation data and a reproducible Ant/NetBeans build and defense workflow.

Implemented feature overview

Area	Implemented capability
Daily finance	Dashboard, balances, accounts, income, expenses, transfers, categories, edit/delete confirmation, search and filters
Planning	Monthly budgets and progress, recurring transactions, quick entry, savings goals, debts and repayments
Analysis	Cash-flow summaries, portfolio analytics, calendar activity, advanced reports and CSV/PDF outputs
Data safety	Atomic CSV writes, backup/restore, validated CSV import, Money Manager preview/import and per-user demo data
Access	Local sign-in, email OTP registration/reset, offline recovery, roles, audit history, isolated workspaces and process lock

Java 25 DESKTOP RUNTIME	47 / 47 TEST ENTRY POINTS	2 DISTRIBUTION JARS	0 FUNCTIONAL BLOCKERS
-----------------------------------	-------------------------------------	-------------------------------	---------------------------------

Scope boundary. Wealthora contains no cloud finance backend. Existing sign-in and normal finance operations remain local. Network access is limited to explicit email-code actions through the standalone OTP relay.

2. Architecture and offline-first data design



The application entry point constructs repositories and services, opens authentication, then creates the finance frame for the verified session. Swing panels manage interaction and presentation; services implement use cases; validators protect domain rules; repositories parse and safely replace local CSV files. This separation keeps GUI classes away from direct file mutation and allows model and service behavior to run headlessly in tests.

Production layers

Layer	Responsibility and examples
Presentation	Swing frames, panels, dialogs, typed table models, listeners, renderers and Java2D charts
Domain	Transaction, Income, Expense, Transfer, Account, MonthlyBudget, RecurringEntry, SavingsGoal and Debt
Services	Finance calculations, reports, import/export, backup, recurring work, presentation data and authentication orchestration
Persistence	Project-local CSV repositories with validated parsing and same-directory safe replacement
OTP boundary	Desktop EmailVerificationGateway plus standalone Java relay for code lifecycle and Gmail STARTTLS delivery

Portable, isolated storage

```

<project-root>/data/
  auth/          local users and audit history
  users/<user-id>/ isolated finance CSV workspace
  backups/       application safety and user backups
  settings/      migration decisions
  presentation/  per-user sample-data manifests
  
```

AppPaths resolves and normalizes the project root, while authentication activates only a trusted generated user identifier. ProjectDataLock owns data/.wealthora.lock so a second process cannot write concurrently. Older OS application data is only an opt-in migration source: existing portable data blocks overwrite, staged files are validated and failed migration rolls back installed files.

Offline-first boundary

Existing-user sign-in, finance CRUD, balances, budgets, analytics, reports, backup/import/export and recovery-question reset do not require the relay. Only explicit send, resend or verify actions use the configured loopback relay. Relay failure therefore produces an OTP-specific error while the finance application remains usable.

3. Object-oriented implementation

The official OOP requirements are part of the real transaction, balance, export and JTable workflows. They are not disconnected demonstration classes.

Requirement	Production class / method	Application use	Automated evidence
Abstraction	abstract Transaction; calculateImpact()	FinanceService and ExportService consume Transaction references	FinanceModelTest
Inheritance	Income extends Transaction; Expense extends Transaction	Both reuse validated transaction state	FinanceModelTest
Polymorphism	Income: +amount; Expense: -amount	Balances, dashboard net flow and export call the override	FinanceModelTest; FinanceServiceTest
Encapsulation	private amount/date; final getters; validated update	Forms/services cannot assign state directly	ExpenseTest; FinanceModelTest
Interface	Exportable.generateCSV(); ExportService implements it	Data > Export Transactions writes generated CSV	ExportServiceTest
Swing JTable	TransactionTableModel + StyledTable	Transactions/Overview display production finance rows	SwingFoundationTest; GUI smoke

Polymorphic impact path

```
Transaction income -> Income.calculateImpact() -> +amount
Transaction expense -> Expense.calculateImpact() -> -amount
FinanceService.calculateBalances():
  for (Transaction item : getAllTransactions())
    apply(balance, item.getAccount(), item.calculateImpact())
```

Transaction keeps amount and LocalDate private. Both are accessed through final getters; BigDecimal and LocalDate are immutable. Tags are copied with List.copyOf. Constructors validate identifiers, positive amounts, posted dates, accounts, categories and text. Expense validates every proposed update before changing shared state, so a failed edit leaves the previous object intact.

Functional export and table path



The GUI receives ExportService from the application wiring. exportTransactions() calls generateCSV(), sorts the production Transaction list and writes type, amount, signed impact, category, account and note with CSV escaping. The same Income and Expense objects are converted into TransactionRow values for the typed JTable. Tests compare generated and written CSV content and confirm export does not mutate repositories.

4. GUI, security, testing and validation

Swing workflow

The main frame provides Dashboard, Transactions, Finance, Budgets, Recurring, Calendar, Planning, Reports, Settings, Profile and Data Management navigation. The transaction JTable supports typed columns, search, account/category/type/date filters, sorting, selection, edit and delete confirmation. Sanitized manual flows covered finance CRUD, totals, transfers, budgets, analytics, presentation data, successful import/export, deletion, restart persistence, user isolation and the exclusive-lock warning.

Authentication and OTP controls

Control	Verified implementation
Local credentials	BCrypt over SHA-256 pre-hash; no plaintext password or recovery answer is persisted
Registration	No user or workspace is saved before successful REGISTRATION verification
OTP lifecycle	Six digits; 10-minute expiry; 60-second cooldown; five attempts; latest code only; replay protection
Binding / limits	Challenge bound to normalized @northsouth.edu email and purpose; email/IP rate limits
Email format	UTF-8 multipart/alternative with plain fallback and branded responsive HTML; code absent from subject/logs
Offline recovery	Protected local recovery question remains independent of email delivery
One-click setup	Gmail/App Password and signing secret remain outside Git as Windows DPAPI CurrentUser ciphertext

Final verification evidence

47 / 47 JAVA ENTRY POINTS	PASS LAUNCHER / DPAPI	PASS CREATE + RESET EMAIL	PASS BUILD + 2 JARS
-------------------------------------	---------------------------------	-------------------------------------	-------------------------------

Using Apache Ant 1.10.17 and Java/Javac 25.0.4, ant clean test-quality jar completed with BUILD SUCCESSFUL. It compiled 265 production and 48 test source files, executed all 47 normal quality-chain entry points, included OTP relay lifecycle/template checks and rebuilt both JARs. Manual evidence confirms branded Create Account and Forgot Password Gmail layouts, resend/older-code rejection, password changes, protected offline recovery, persistence, two-user isolation, relay-unavailable messaging and lock presentation.

Build and run

```
ant clean test-quality jar
java -jar dist\Wealthora.jar
```

Windows demo: configure once with Configure Wealthora OTP.cmd, then use Start Wealthora.cmd. For NetBeans F6, keep Start OTP Relay for NetBeans.cmd open. The private persistence gate confirmed both launch paths reuse the saved encrypted configuration without another credential prompt.

5. Conclusion, contributions and future scope

Conclusion

Wealthora satisfies the CSE 215 Personal Expense Tracker objective as a substantial Java Swing application. Required OOP concepts drive production transaction calculations, CSV export and JTable presentation. The local/offline-first boundary keeps finance records inside isolated user workspaces, while the separately configured relay adds verified email ownership and password reset without receiving passwords or finance data. The final verified release has no known functional blocker.

Team contributions

Member	Final allocation
Shibli Rahman Moon 2534187012	Project Lead, System Design & Integration: professional UI/dashboard; authentication and email OTP; password recovery; multi-user integration; overall integration; QA and 47/47 verification; final build; documentation coordination; presentation/live-demo preparation
Md. Nafij Jaman Rabbi 2513403642	Core Finance Modules: accounts; income/expense; transfers; budgets; goals; debt management; finance validation; related functional testing
Md. Monimul Haque 1821781042	Reports & Productivity Modules: reports; recurring transactions; calendar; import/export; backup/restore; demo-data support; related testing

Limitations

- Finance data is intentionally local to one project copy; cross-device synchronization is not implemented.
- Relay restart invalidates pending in-memory challenges; the current design targets one local/demo instance.
- Real email delivery depends on controlled Gmail/Google Workspace configuration and network availability.
- Windows DPAPI and offline voice recognition are platform-specific; CSV storage relies on the exclusive writer lock.

Future improvements

- Optional encrypted workspace export and explicit conflict-aware synchronization.
- Durable shared OTP challenge storage and provider-neutral mail adapters.
- More accessibility testing, locale-aware formatting, forecasting and additional import formats.

Concise references

- Wealthora source: model, service, repository, ui, auth and otp-relay source trees.
- Project mapping: docs/OOP_MAPPING.md, docs/ARCHITECTURE.md, README.md and build.xml.
- Oracle Java SE 25 API documentation (java.desktop / Swing / JTable).
- Apache Ant 1.10 manual and the checked NetBeans Ant project metadata.

Demonstration reminder. Carry a printed hard copy of this six-page report on August 25, 2026. Use the configured project laptop for the live OTP path and retain the verified offline demo path.